

CSC 720: Electronic Commerce Technologies

Implementation Examination Questions — Flet Marketplace Application

Modibbo Adama University, Yola

Faculty of Computing · Department of Computer Science

M.Sc. / PGD Computer Science Programme

*Technical questions on the design, implementation, security, testing
and deployment of the MAUMart marketplace application*

Scope and How to Use This Document

This is a companion to the course-wide question bank and covers different ground. Where that document examines the lecture material and the design rationale behind the project, this one examines the implementation itself: the Flet client architecture, the FastAPI backend, the data model, the business logic of checkout and orders, the recommendation engine, the security controls, the testing strategy, and the deployment and scalability position.

These are the questions a technically engaged examiner asks once they have read the report and opened the code. They are harder to prepare for by memorisation than theory questions, because the examiner can point at a specific file and ask why it does what it does. The answers here are therefore written to show reasoning against the actual implementation rather than to recite general principles.

A recurring feature of the answers is that they concede limitations explicitly — the stock race condition, the in-process session store, the unverified container and CI configuration, the structural-only client testing. This is deliberate. In a viva the examiner has usually already found the weaknesses, and a candidate who identifies them first, characterises them accurately and states the remedy is substantially more credible than one who defends work that does not withstand scrutiny. Every limitation named here is real; none should be hidden.

If your submitted implementation differs from the one described, adapt the specifics. The structure of each answer — what was done, why that choice, what it costs, what it fails to cover — transfers regardless of the details.

Contents

Section A: The Flet Framework and Client Architecture — Q1–Q6

Section B: Backend Architecture and API Design — Q7–Q12

Section C: Data Model and Persistence — Q13–Q18

Section D: Business Logic — Checkout, Orders and Payments — Q19–Q24

Section E: The Recommendation Engine — Q25–Q28

Section F: Security Implementation — Q29–Q33

Section G: Testing and Quality Assurance — Q34–Q37

Section H: Deployment, Scalability and Limitations — Q38–Q43

Total: 43 questions with full answers.

Section A: The Flet Framework and Client Architecture

Q1. What is Flet, and by what mechanism does a single Python codebase produce Desktop, Web and Mobile applications?

Answer. Flet is a Python framework that wraps Flutter, Google's cross-platform UI toolkit. The developer writes Python that constructs a tree of Flet controls (Text, Column, Row, Container, ElevatedButton and so on); Flet serialises that control tree and dispatches it to a Flutter client, which performs the actual rendering. Event callbacks travel back from the Flutter client to the Python process over the same channel.

The single-codebase property therefore comes from Flutter rather than from Python. Flutter compiles a single widget description to native code on Android and iOS, to desktop binaries on Windows, macOS and Linux, and to a canvas-based web target in the browser. Flet exposes that capability to Python without requiring the developer to write Dart.

The architectural consequence that matters for examination purposes is that the Python process is not compiled into the client. It runs as a separate process hosting the application logic, with the Flutter layer acting as a rendering and input surface. This is why `flet run main.py --web`` starts a server rather than producing a static bundle, and why the same code can drive a desktop window or a browser session unchanged.

In MAUMart this yields the deployment matrix the assignment required: `flet run main.py`` for a desktop window, `flet run main.py --web`` for browser access, and `flet build apk`` or `flet build ipa`` as packaging steps over identical source rather than separate ports.

Q2. Justify the choice of Flet over Flutter/Dart, React Native, or a conventional web stack for this project.

Answer. The decisive argument is team capability rather than framework superiority. A Computer Science cohort already fluent in Python can be productive in Flet immediately, whereas Flutter requires learning Dart and React Native requires JavaScript plus its own ecosystem. For a time-boxed coursework project, the cost of a second language is paid in weeks that the schedule does not have.

Flet also satisfies the assignment's explicit constraint of one codebase across three targets, which a conventional web stack does not: a React or Django application gives web, and then requires a separate effort for native mobile.

The honest counter-arguments belong in a complete answer. Flet is a younger and smaller ecosystem than Flutter or React Native, so fewer components, examples and Stack Overflow answers exist, and its API is still changing between versions — the 0.86 release used here differs materially from earlier releases in control constructors. Flutter/Dart gives access to the full widget catalogue and plugin ecosystem that Flet exposes only partially.

Most significantly, Flutter's web target renders to canvas rather than semantic HTML, so a Flet web page is effectively invisible to search-engine crawlers. For a marketplace, where organic discovery is a primary acquisition channel, this is a genuine defect rather than a minor limitation — and it is why MAUMart adds a separate server-rendered storefront rather than relying on the Flet web build alone.

Q3. Describe how state is managed in the MAUMart Flet client, and explain why a dataclass was used rather than a global variable or per-view state.

Answer. Client state is held in a single ``AppState`` dataclass in ``state.py``, holding the API base URL, the authentication token, the user's identifier, name and role, and the cart as a list of ``CartLine`` objects. A single instance is created in ``main()`` and passed into every view builder together with the page object and the navigate function.

The dataclass was chosen over a bare global because it gives a declared schema with type hints, default values via ``field(default_factory=...)``, and methods that keep related invariants together — ``add_to_cart``

increments an existing line rather than duplicating it, ``remove_from_cart`` filters by listing identifier, ``cart_total`` computes from lines, and ``logout`` clears authentication and cart atomically. Scattering that logic across views would allow one view to clear the token but leave a stale cart.

Per-view state was rejected because the cart and session must survive navigation. ``navigate()`` calls ``page.clean()`` and rebuilds the view from scratch, so any state local to a view is destroyed on every transition; only state held in the shared object persists.

One design point worth defending explicitly: the ``api_base`` field reads ``MAUMART_API_BASE`` from the environment through ``default_factory``. This was originally hardcoded to localhost and had to be changed when containerisation revealed that the backend is reachable by service name rather than ``127.0.0.1`` inside a Docker network — an illustration that configuration which varies by deployment environment does not belong in source.

The limitation to concede: this state is in-memory and per-process, so it is lost on restart and is not shared across instances. Persisting the token to secure client storage would be the next step for a real application.

Q4. Explain the navigation mechanism in the MAUMart client and evaluate its design.

Answer. ``main.py`` defines a single ``navigate(route, **kwargs)`` closure over the page and application state. It calls ``page.clean()`` to remove existing controls, dispatches on the route string to the appropriate view builder — ``auth_view.login_view``, ``home_view.build``, ``product_view.build`` and so on — appends the returned control tree with ``page.add(view)``, and calls ``page.update()``. Every view receives ``navigate`` itself, so any control can trigger a transition by calling it.

Additional arguments pass through ``kwargs``, which is how the product route receives its ``listing_id``. Unrecognised routes fall through to the home view, so a bad route degrades rather than crashing.

Strengths: it is simple, has no framework dependency, and makes the full route table visible in one function. Role gating is enforced inside the view builders themselves — ``vendor_view`` and ``admin_view`` redirect to home if the role does not match — so authorisation is not scattered through the navigation function.

Weaknesses that a candid evaluation should state. There is no browser history integration, so the back button does not work as a user expects in the web build. ``page.clean()`` followed by a full rebuild discards and reconstructs the entire control tree on every transition, which is wasteful and loses scroll position and transient form input. There is no route parameter validation, so a malformed identifier surfaces as an API error rather than being caught early. And client-side role checks are a usability convenience only — the authoritative enforcement is the backend's ``require_role`` dependency, which is what actually prevents a crafted request from reaching a privileged endpoint.

Q5. The client communicates with the backend exclusively through an ``api_client`` module. What principle does this follow and what does it buy?

Answer. It follows separation of concerns, and specifically the principle that infrastructure detail should be isolated behind an interface rather than spread through presentation code. No view in MAUMart imports ``requests``; every view calls a named function such as ``api_client.browse_listings(state, ...)`` or ``api_client.checkout(state, items, address, method)``.

What it buys, concretely. First, uniform authentication: the ``_headers`` helper attaches the bearer token from application state to every authenticated call, so no view can forget it. Second, uniform error handling: ``_handle`` inspects the status code, extracts the ``detail`` field from a FastAPI error body, and raises a typed ``ApiError``, so views catch one exception class rather than parsing HTTP semantics individually.

Third, changeability. Adding retries with exponential backoff, request timeouts, a circuit breaker, or request logging is a change to one module rather than to nine view files. Swapping the transport entirely

— to a WebSocket channel or a generated client from an OpenAPI specification — likewise touches one file.

Fourth, testability: the module is the natural seam at which to substitute a fake backend for testing views in isolation, though MAUMart's smoke test in fact exercises the real API rather than mocking it, which trades isolation for integration realism.

The general form of the principle is the dependency rule: presentation depends on an abstraction of data access, not on the concrete HTTP library. This is the same reasoning that puts business logic behind the application tier rather than in the client, one layer up.

Q6. Why does the Flet client not connect directly to the database, given that Python can do so trivially?

Answer. Because that would be the two-tier fat-client architecture described in Unit 7.2.1, and it fails on three counts that matter for an open Internet application.

Security is the decisive one. A client that connects directly to a database must hold database credentials, and any credential shipped to a client is compromised — it can be extracted from the package, from memory, or from network traffic. The database would additionally need to be reachable from the public Internet, exposing it directly to attack rather than shielding it behind a logic tier. Authorisation would also have to be enforced by database grants, which cannot express rules like 'a vendor may update only orders belonging to their own vendor record'.

Scalability is the second. Every client would hold a database connection, so connection count scales with concurrent users and exhausts the connection pool long before compute becomes the constraint.

Maintainability is the third. Business rules embedded in the client are coupled to client releases: a change to the order status state machine or the VAT rate would require every installed desktop and mobile client to be updated, and old clients in the field would continue applying stale rules to shared data.

The three-tier arrangement MAUMart uses instead places business rules in the FastAPI application tier, shields the database behind it, keeps the application tier stateless so it scales horizontally, and allows the same rules to serve the Flet client and the server-rendered storefront without duplication.

Section B: Backend Architecture and API Design

Q7. Describe the backend architecture of MAUMart, identifying each layer and its responsibility.

Answer. The backend is a three-tier arrangement implemented in FastAPI. The presentation boundary is the HTTP API itself: routers under ``routers/'` expose resource-oriented endpoints — ``/auth``, ``/listings``, ``/cart``, ``/orders``, ``/reviews``, ``/recommendations``, ``/interactions``, ``/admin``, ``/users`` — plus ``/store`` serving the server-rendered HTML storefront from Jinja2 templates.

Request and response contracts are declared as Pydantic models in ``schemas.py``. These are not merely documentation: FastAPI validates every incoming body against them and rejects malformed input with a 422 before any handler runs, which is boundary validation acting as a security control as described in Unit 8.2.1.

The application tier is the router handlers themselves, holding business rules — the checkout algorithm, the order status state machine, the recommendation tiering, the vendor verification gate. Cross-cutting concerns are factored out: ``auth_utils.py`` for hashing and session handling, ``rate_limit.py`` for throttling, ``notifications.py`` for outbound messaging, ``payment_gateway.py`` for payment.

The data tier is SQLAlchemy ORM models in ``models.py`` mapping to the entities of the ERD, with ``database.py`` supplying the engine and session factory and ``get_db`` providing a per-request session through FastAPI's dependency injection.

Middleware wraps the whole: a request-logging middleware emitting structured JSON, a security-headers middleware, and CORS with an explicit origin allow-list. The layering is what allows the Flet client and the storefront to share one set of business rules rather than duplicating them.

Q8. Explain FastAPI's dependency injection as used in this project, with reference to ``get_db``, ``get_current_user`` and ``require_role``.

Answer. Dependency injection in FastAPI declares what a handler needs as a parameter with a ``Depends(...)`` default; the framework resolves and supplies it before the handler runs. Three uses in MAUMart illustrate three distinct purposes.

``get_db`` is a generator dependency supplying a database session. It yields a session and closes it in a ``finally`` block, so the session lifetime is bound to the request and closure is guaranteed even if the handler raises. This is resource management expressed declaratively rather than as boilerplate in every handler.

``get_current_user`` is an authentication dependency. It reads the ``Authorization`` header, strips the bearer prefix, looks the token up in the session store, checks expiry, loads the corresponding user and returns it — raising 401 at any failure point. A handler that declares ``user: models.User = Depends(get_current_user)`` is therefore guaranteed an authenticated user, and cannot accidentally omit the check.

``require_role`` is an authorisation dependency and is a factory rather than a dependency itself: ``require_role('vendor')`` returns a checker closure that depends on ``get_current_user`` and raises 403 if the role does not match. This composition is the point — authorisation builds on authentication without repeating it.

Two further benefits worth naming: dependencies appear in the generated OpenAPI documentation, and they can be attached at router or route level via the ``dependencies=[...]'` parameter, which is how rate limiting is applied to the auth endpoints without touching their signatures.

Q9. Why was SQLAlchemy ORM used rather than raw SQL, and what is the security implication?

Answer. Three reasons, of which the security one is decisive for an open Internet application.

Security: the ORM emits parameterised queries, binding values separately from statement text. This structurally prevents SQL injection, because user input is never concatenated into a statement that the

database then parses. A hand-written string-concatenated query is the canonical injection vulnerability and appears in the OWASP Top 10; the ORM removes the opportunity to write one by default rather than relying on developer discipline.

Portability: because queries are expressed against mapped classes rather than dialect-specific SQL, the same code runs against SQLite in development and PostgreSQL in production. In MAUMart this claim was tested rather than assumed — the full test suite was run against both backends, with only the `DATABASE_URL` environment variable changed.

Maintainability: relationships are declared once on the models (`Order.items`, `Listing.vendor`, `Vendor.listings`) and traversed as attributes, so joins are expressed by navigating objects rather than by repeating join clauses in every query.

The honest limitation: ORMs abstract away the generated SQL, which invites the N+1 query problem — iterating a collection and lazily loading a relation per element. MAUMart's collaborative filtering routine, which loads all confirmed orders and iterates their items, is the place this would bite first at scale, and the remedy is eager loading with `joinedload` or `selectinload` rather than abandoning the ORM.

Q10. The project claims that switching from SQLite to PostgreSQL is a one-line change. Is that claim accurate?

Answer. Largely, but not entirely, and the qualification is the substance of a good answer.

What is accurate: `database.py` reads `DATABASE_URL` from the environment, defaulting to SQLite. Setting it to a `postgresql://` URL is sufficient to seed and run the full application against PostgreSQL, and the entire test suite passes unchanged against both. The `connect_args` containing SQLite's `check_same_thread` flag is applied conditionally, since it is meaningless to other drivers.

What the claim glosses over. First, a driver must be installed — `psycopg2` is required for PostgreSQL and is not needed for SQLite. Second, `seed.py` branches on dialect: for SQLite it deletes the database file, which does not exist for a server-based database, so for PostgreSQL it drops and recreates tables instead. Third, `listings.py` deliberately branches on `engine.dialect.name` to use PostgreSQL full-text search (`to_tsvector`/`plainto_tsquery` with `ts_rank`) rather than the `ILIKE` scan SQLite falls back to — a deliberate exploitation of a capability difference rather than a portability failure.

Fourth, and generally true beyond this project: the two systems differ in type affinity, transaction isolation and concurrency behaviour, so a migration requires re-running concurrent paths under the new engine rather than only re-pointing a URL. Note that this project's own stock race was reproduced on SQLite itself, not merely expected on PostgreSQL (see the race-condition question in Section D).

The defensible formulation is that the ORM makes the query layer portable, which is the hard part, while configuration, seeding and dialect-specific optimisation remain deliberate work.

Q11. Explain the request-logging and security-headers middleware, and justify the decision not to set HSTS.

Answer. Both are registered with FastAPI's `@app.middleware('http')` decorator, which wraps every request. The logging middleware records a start time, awaits the downstream handler, computes elapsed milliseconds, and emits a single JSON line containing method, path, status code, duration and client address — escalating to warning level for 5xx responses and logging a full traceback on an unhandled exception before re-raising it.

JSON rather than free text is the significant choice: structured lines are machine-parseable by any log aggregator, filterable and aggregable without regular expressions. A production incident can be investigated by querying fields rather than by reading prose.

The security-headers middleware sets three headers on every response. `X-Content-Type-Options: nosniff` prevents browsers from MIME-sniffing a response into a different content type, which blocks a class of attack where uploaded content is interpreted as script. `X-Frame-Options: DENY` prevents the

application being embedded in a frame, defeating clickjacking. `Referrer-Policy: strict-origin-when-cross-origin` limits how much URL information leaks to third-party destinations.

HSTS is deliberately omitted, and the reasoning is important. `Strict-Transport-Security` instructs a browser to refuse plain HTTP to that host for the declared duration, and browsers cache that instruction persistently. Setting it on an application that serves plain HTTP in development and coursework use would lock developers and users out of their own service, with no easy remedy short of clearing browser state. The correct placement is at the reverse proxy that actually terminates TLS in production — which is where the architecture assumed TLS termination would occur in any case.

Q12. What is CORS, why was the original wildcard configuration wrong, and what replaced it?

Answer. Cross-Origin Resource Sharing is a browser-enforced mechanism governing whether script running on one origin may read responses from another. The browser sends the requesting `Origin` and, for non-simple requests, first issues a preflight `OPTIONS` request; the server replies with `Access-Control-Allow-Origin` and related headers, and the browser permits or blocks access accordingly.

The original configuration used `allow\_origins=['\*']`, permitting any website to issue authenticated cross-origin requests to the API and read the responses. The consequence is that a malicious page a user happens to visit could call MAUMart endpoints from the user's browser. Because MAUMart authenticates with a bearer token in a header rather than with cookies, the exposure is narrower than in a cookie-session application — the attacker's page cannot silently attach the token — but a wildcard remains indefensible in production, and combining it with credentialed requests would be worse still.

It was replaced with an explicit allow-list read from the `MAUMART\_ALLOWED\_ORIGINS` environment variable, defaulting to the local Flet development ports. This was verified behaviourally rather than assumed: a preflight from an allowed origin returns 200 with the origin echoed in the response header, while a preflight from an arbitrary origin is refused.

The point worth generalising is that CORS is a browser protection, not a server authorisation mechanism. It does not stop a direct HTTP client such as `curl` from calling the API, so it supplements authentication and authorisation rather than substituting for them.

Section C: Data Model and Persistence

Q13. Why are Users and Vendors modelled as separate tables in an optional one-to-one relationship rather than as a single table?

Answer. Because vendor-specific state exists only for a subset of users, and folding it into the user table would require nullable columns whose meaning depends on the role — ``business_name``, ``category`` and ``verification_status`` are meaningless for a buyer.

The separation gives three concrete benefits. First, integrity: a vendor row exists if and only if the user has registered to sell, so the presence of the row is itself the fact, rather than a nullable column that could be partially populated. Second, a single account can hold both roles naturally — a student who buys textbooks and also sells them is one user with one vendor profile, not two accounts.

Third, the foreign key from ``Listing.vendor_id`` points at the vendor record rather than at the user, which correctly expresses that listings belong to a business identity. When account deletion anonymises the user, the vendor's business name is separately anonymised and its listings deactivated, without the two concerns being entangled in one row.

The relationship is declared in SQLAlchemy with ``uselist=False`` on the user side, marking it one-to-one rather than one-to-many, and ``unique=True`` on ``Vendor.user_id`` enforces that at the database level rather than only in application code. Enforcing the constraint in the schema rather than in a handler is the defensible pattern, because the database is the last line and application code can be bypassed by a migration script or a second service.

Q14. A single checkout across multiple vendors creates several Order rows sharing a session identifier. Justify this over one order with vendor-tagged line items.

Answer. Because the order is the unit of fulfilment, and fulfilment is per vendor. Each vendor confirms, prepares and delivers independently, progresses through the status state machine at their own pace, is paid separately, and is disputed against separately. Modelling one order spanning vendors would force every one of those concerns to be tracked per line item instead, duplicating the state machine at the wrong granularity.

Concretely: ``Order.status`` is a single column advancing through ``confirmed_paid`` → ``confirmed`` → ``ready_for_delivery`` → ``completed``. If one order contained items from three vendors, that column could not represent the common case where one vendor has delivered and another has not yet confirmed. ``Review`` links to an order and constrains one review per completed order, which is coherent only if the order has exactly one vendor. Vendor payout and refund likewise operate on an order total.

The ``session_id`` preserves what the buyer experienced. All orders created by one checkout share it, so the buyer's view can group them as a single transaction, and the payment gateway is charged once for the combined total using that identifier as the idempotency key.

The trade-off to acknowledge: a refund of one vendor's order within a multi-vendor session is a partial refund of a single gateway charge, which requires the gateway to support partial refunds and requires reconciliation between the session-level charge and the order-level refunds. Storing ``payment_reference`` on each order rather than only on the session is what makes that tractable.

Q15. Why does ``OrderItem`` store ``price_at_purchase`` rather than joining to the listing's current price?

Answer. Because an order is a historical record of a completed transaction, and a transaction record must not change when unrelated data changes afterwards.

If the order displayed the listing's current price, then a vendor reducing the price next week would retroactively alter what a past order appears to have cost. The buyer's receipt would no longer match what they paid, the platform's revenue reporting would shift under it, and any tax or audit record derived from it would be wrong. If the listing were deleted entirely, the order would lose its price altogether.

This is an instance of the general distinction between transactional and reference data. Reference data — the current catalogue — is mutable and represents the present. Transactional data records what happened at a point in time and must be immutable once written. Denormalising the price into the order item is therefore not redundancy to be eliminated but a deliberate snapshot.

The same reasoning drives the sibling decision to store `subtotal`, `vat\_amount` and `delivery\_fee` as recorded columns on the order rather than recomputing them from a current VAT rate. If Nigeria's VAT rate changed, every historical order would otherwise silently misreport its tax component — which for a financial record is not a display bug but a compliance failure.

Q16. Explain the purpose of the `Interaction` entity and why recommendation signal is not derived from `Order` alone.

Answer. `Interaction` is an append-only log of user behaviour against listings, storing user, listing, type (`view` or `purchase`) and timestamp. It exists because purchases alone are too sparse and too narrow a signal for the recommendation engine.

Sparsity first: most users purchase rarely, so a system depending only on orders has almost no data about most users and cannot leave the cold-start tier. Views are orders of magnitude more numerous and arrive immediately, so a user crosses the three-interaction threshold and receives personalised results within a single session.

Signal breadth second: a view expresses interest that a purchase does not capture. A user who views five laptop bags and buys none has revealed a clear preference; an order table records nothing. Conversely, purchases carry stronger confidence, which is why implicit-feedback methods weight interaction types differently rather than treating them as equivalent.

Keeping the log separate also serves a governance purpose that a derived view could not. Because the recommendation signal lives in its own table, the user-facing control that clears recommendation history can delete from it without touching orders — implementing objection to profiling under NDPA without destroying transaction records that carry an independent retention obligation. Had the signal been derived from orders, those two rights would have been impossible to separate.

The scalability caveat: an append-only behaviour log grows without bound and would in production need partitioning, retention limits and aggregation into precomputed features rather than being queried directly.

Q17. How is the verified-purchase review constraint enforced, and why is schema design part of that enforcement?

Answer. `Review` carries a foreign key to `Order` rather than merely to a buyer and a vendor. The handler then enforces three conditions: the order must exist and belong to the requesting user, its status must be `completed`, and no review may already exist for that order.

Schema design is part of the enforcement because the shape of the table determines what is expressible. With a review linked only to a buyer-vendor pair, there is no row-level fact against which 'this reviewer actually completed a transaction with this vendor' can be checked, and any enforcement would depend entirely on application logic that a second code path could bypass. With the order foreign key present, the constraint is anchored to a specific transaction that either exists or does not.

The theoretical motivation is from Unit 4.6: reputation systems are attacked by ballot stuffing, in which fabricated or colluding ratings inflate a score. Restricting reviews to verified completed purchases raises the cost of the attack to the cost of actually transacting, which is the standard countermeasure.

Residual weaknesses to concede. A determined attacker can still place and complete low-value self-dealing orders to manufacture reviews, so the constraint raises cost rather than eliminating the attack; detecting that pattern requires graph analysis of buyer-seller pairs, which MAUMart does not implement. The uniqueness of one review per order is also enforced by a query in the handler rather than by a

database unique constraint, which is weaker than it should be — a concurrent double submission could in principle pass both checks.

Q18. Password reset and email verification tokens are stored as hashes rather than as raw values.

Explain why, and how the flows work.

Answer. The tables `PasswordResetToken` and `EmailVerificationToken` store `token_hash`, an expiry timestamp and a `used` flag, never the raw token. The reasoning is the same as for passwords: a reset token is a credential, because possession of it permits taking over an account. An attacker with read access to the database — through SQL injection, a leaked backup, or an insider — could otherwise reset any account at will.

Hashing means the stored value is not usable as a credential. The raw token exists only in transit to the user's email and in the request that redeems it, at which point the server hashes the submitted value and compares against the stored hash. SHA-256 is adequate here rather than bcrypt because the token is high-entropy and randomly generated, so the slow-hashing that protects low-entropy human passwords against brute force is unnecessary.

The reset flow: a request generates a six-digit OTP, stores its hash with a fifteen-minute expiry, and dispatches the raw value through the notification backend. Confirmation checks the hash, the unused flag and the expiry, sets the new password, marks the token used, and — importantly — invalidates every active session for that user, so a reset performed because of a suspected compromise actually evicts the attacker.

One security property worth highlighting: the request endpoint returns an identical generic response whether or not the email is registered. Returning 'no such account' would let an attacker enumerate valid email addresses, which is a reconnaissance vector for credential stuffing. The endpoint is additionally rate-limited.

Section D: Business Logic — Checkout, Orders and Payments

Q19. Walk through the checkout algorithm as implemented, identifying where each failure mode is handled.

Answer. Step one, validation: each requested item is loaded and checked for status `'live'` and sufficient stock. Failure raises 400 with the offending item named rather than silently dropping it, because a cart that quietly shrinks is a usability failure and, at checkout, a trust failure.

Step two, grouping: items are grouped by `'vendor_id'` using a `defaultdict`, since the order is the per-vendor fulfilment unit.

Step three, costing: per group the subtotal is summed from stored listing prices, VAT is computed at 7.5% of that subtotal, and a flat delivery fee is added. The per-group total accumulates into a session total. The client has already displayed this breakdown before enabling the confirm control, which is the dark-pattern avoidance requirement from Unit 2.7 rather than a presentational choice.

Step four, persistence: a `'pending_payment'` order is created per vendor group with its subtotal, VAT, delivery fee and total recorded as separate columns, along with order items carrying `'price_at_purchase'`. All share a generated `'session_id'`. This is committed before the gateway is called, so that a crash mid-payment leaves a recoverable record rather than a charge with no corresponding order.

Step five, payment: the gateway is charged once for the session total with `'session_id'` as the idempotency key. On success each order is marked `'confirmed_paid'`, the payment reference is stored, stock is decremented, purchase interactions are logged, and buyer and vendor notifications are dispatched. On failure the orders are marked `'payment_failed'` and a 402 is raised.

The weakness to concede: the stock check and the decrement are not atomic, which is the subject of the next question.

Q20. Identify the race condition in the stock handling and explain how you would correct it.

Answer. The defect is a time-of-check to time-of-use gap. Availability is validated at step one, but the decrement happens at step five after the gateway round trip. Two concurrent checkouts for the last remaining unit can both pass validation, both succeed at payment, and both decrement — overselling the item and driving stock negative, since the decrement is an unguarded read-modify-write on `'listing.quantity'`.

SQLite does not protect against this, and assuming it would is the mistake. Measured with three simultaneous checkouts for a single remaining unit, all three buyers were charged and received HTTP 200 in every one of three trials, leaving stock at -1 or 0 through lost updates. FastAPI runs synchronous handlers in a thread pool and the availability check and the decrement are separate statements, so the window is open on any engine. The defect survived development for a simpler reason: none of the twenty-nine functional test cases ran two checkouts at once. PostgreSQL's default read-committed isolation would be expected to behave the same way for an unguarded read-modify-write, but that has not been measured. The lesson is that concurrency defects are found by concurrency tests.

Three corrections, in ascending robustness. First, pessimistic locking: select the listing rows `'FOR UPDATE'` within the transaction, so a second transaction blocks until the first commits. Simple and correct, but it serialises access to popular items and holds locks across the gateway call unless the transaction is restructured.

Second, an atomic conditional update: issue `'UPDATE listings SET quantity = quantity - :n WHERE id = :id AND quantity >= :n'` and treat zero affected rows as insufficient stock. This is a single atomic statement with no lock held across the network call, and is the most proportionate fix for this system.

Third, reservation: decrement into a held state at validation time with a timeout that releases the hold if payment does not complete, which is what high-contention flash-sale designs use — frequently backed by an atomic counter in Redis rather than the relational row.

The correct sequencing change accompanying any of these is to reserve before charging rather than after, so that a customer is never charged for stock the system cannot supply.

Q21. Explain idempotency in the payment flow, why it is necessary, and how it is implemented here.

Answer. An operation is idempotent if performing it more than once has the same effect as performing it once. Payment must be idempotent because the network between client, server and gateway is unreliable: a request may succeed at the gateway while the response is lost, a client may retry after a timeout, and a user may double-click a confirm button. Without idempotency each of those produces a second charge against a customer who authorised one.

The implementation generates a ``session_id`` UUID per checkout and passes it as the idempotency key to ``payment_gateway.charge``. The mock gateway keeps a dictionary of processed keys and returns the stored result verbatim on a repeat, so a replayed call produces no second charge. A real Paystack integration achieves the same by passing the key as the transaction ``reference``, which the provider rejects as a duplicate.

The property is required on the inbound side as well, and this is the part candidates usually omit. Gateways deliver webhooks with at-least-once semantics and retry on any non-2xx or timeout, so the same ``charge.success`` event will arrive more than once. A consumer must therefore key its processing on the transaction reference and make repeat delivery a no-op rather than confirming the order and decrementing stock a second time.

The related discipline is that the webhook is a notification rather than the source of truth: on receipt, or on timeout with no receipt at all, the server should call the gateway's verify endpoint to establish authoritative status before releasing goods.

Q22. Explain the order status state machine and why transitions are validated against a table rather than simply assigned.

Answer. ``ORDER_STATUS_FLOW`` in ``models.py`` maps each status to the list of statuses legally reachable from it: ``pending_payment`` to ``confirmed_paid`` or ``payment_failed``; ``confirmed_paid`` to ``confirmed``; ``confirmed`` to ``ready_for_delivery``; ``ready_for_delivery`` to ``completed``; and ``completed`` and ``payment_failed`` to nothing. The update handler checks membership in that list and rejects anything else with 400, returning the permitted next states.

Validating against an explicit table rather than assigning freely matters because status is not a label but a claim about the real world with consequences attached. ``completed`` unlocks the buyer's ability to review. ``confirmed_paid`` is the state in which stock has been decremented. Allowing arbitrary assignment would permit a vendor to mark an order completed without it ever having been confirmed, unlocking a review for a transaction that did not occur, or to move an order backwards and re-trigger downstream effects.

Encoding the machine as data rather than as nested conditionals also gives two practical benefits: the legal transitions are stated in one readable place rather than distributed through handler logic, and the error response can tell the caller what is actually allowed from the current state, which is a usability property as much as a correctness one.

Authorisation is layered on top: the handler first confirms the order belongs to the requesting vendor, so a valid transition applied to someone else's order is rejected with 403 before the transition table is consulted at all. Buyer-initiated cancellation is deliberately confined to ``pending_payment``; after that the dispute process applies, because a confirmed order involves a counterparty who may already have incurred cost.

Q23. How is VAT handled, and what would change if the rate were amended?

Answer. ``VAT_RATE`` is defined as a module-level constant of 0.075 in ``models.py``, matching Nigeria's standard rate. At checkout it is applied to the items subtotal — not to the delivery fee in this

implementation — rounded to two decimal places, and stored in the order's `vat\_amount` column alongside `subtotal`, `delivery\_fee` and `total` as separate recorded values.

Storing the computed amount rather than only the rate is the substantive design decision. If the rate changed and orders recomputed tax from the current constant, every historical order would silently misreport its tax component. For a financial record that is a compliance failure, not a display inconsistency. Recording the amount at the time of the transaction makes each order self-describing and auditable.

What would change on amendment: only the constant, for orders placed afterwards; existing rows are unaffected by design, which is the property being sought. What the current implementation does not support, and should be conceded, is a rate that varies by date or by product category — a correct implementation would hold rates in a table with effective-from dates and a category mapping, and resolve the applicable rate at transaction time rather than reading a single constant.

Two further real-world gaps: whether VAT applies to the delivery fee, and whether a marketplace collects VAT on the goods or only on its own commission, are jurisdiction-specific questions this implementation does not attempt to answer.

Q24. Describe how refunds are implemented and why a status label alone would be inadequate.

Answer. A buyer raises a dispute against an order, creating a `Dispute` row in status `open` that surfaces on the admin queue. The admin resolution endpoint takes an explicit boolean. If false, the dispute becomes `resolved\_no\_refund` and the order is untouched. If true, the handler retrieves the order's stored `payment\_reference`, calls `payment\_gateway.refund` with it and the order total, and only on a successful gateway response sets the order to `refunded` and the dispute to `refunded`, then notifies the buyer. A gateway failure raises 502 and leaves both records unchanged.

A status label alone would be inadequate because money would not move. The customer would see an order marked refunded while their bank account showed the original charge, which is the worst possible failure mode: the system's record and reality diverge, the customer believes they have been refunded, and the discrepancy is discovered only when they complain again.

The ordering is deliberate and worth stating: the external effect is attempted first and the local state is changed only on confirmed success. Reversing that order would produce records claiming a refund that never occurred. This is the same discipline as the payment path, where the order is persisted before charging so that no charge exists without a corresponding record.

The limitation to concede: `payment\_reference` is stored per order, so a refund of one order within a multi-vendor session is a partial refund against a session-level charge. The mock gateway accepts this without complaint; a real integration would need to confirm the provider supports partial refunds against the reference and to reconcile the session total against the sum of order-level refunds.

Section E: The Recommendation Engine

Q25. Describe the three-tier recommendation engine as implemented, including the thresholds that select between tiers.

Answer. The engine dispatches on two counts rather than one, which is the design's main subtlety. It first counts the requesting user's interactions. If that is below `COLD_START_INTERACTION_THRESHOLD`` (three), it returns `_trending`` — the top listings by `sales_count``, requiring no personal history whatsoever — tagged 'Trending'.

Otherwise it counts platform-wide interactions. If that total is below `CF_MIN_PLATFORM_INTERACTIONS`` (twenty), it returns `_content_based``, tagged 'Because you viewed similar items'. That routine builds a profile from the categories and mean price of listings the user has interacted with, scores every other live listing by category match (weighted 2.0) plus price-band proximity, and returns the top results.

Only when both thresholds are cleared does it use `_item_based_cf``. That routine builds a co-occurrence map from all `confirmed_paid`` orders, counting pairs of listings purchased together; retrieves the user's twenty most recent interactions; accumulates neighbour scores weighted by recency as `1/(1+age_days)``; excludes listings the user has already seen and anything out of stock; and returns the top remainder, tagged 'Customers who bought this also bought'.

Two fallbacks guard the edges: if collaborative filtering produces no scores at all it degrades to content-based, and if content-based finds no history it degrades to trending. `TOP_N`` caps every path at eight results.

The reason for the two distinct counts is that a user can be data-rich on a data-poor platform. Personalising from an individual's history is viable long before item-item similarity computed across the whole user base is statistically meaningful, and conflating the two conditions would either withhold personalisation too long or apply collaborative filtering to noise.

Q26. Why was item-based collaborative filtering chosen over user-based, and what are the implementation's scalability limits?

Answer. Item-based was chosen following Sarwar et al. (2001) and Linden et al. (2003), the latter describing Amazon's production approach. The argument is stability: item-item similarity changes slowly, because the set of people who bought both a textbook and a calculator shifts gradually, whereas a user's own interaction vector changes with every session. Stable similarities can be precomputed and cached; volatile ones cannot. Item-based similarity is also explainable in a form users accept — 'customers who bought this also bought' — where 'users similar to you' is both vaguer and more unsettling.

In a catalogue-oriented marketplace the item count is also typically smaller and more stable than the user count, so the similarity matrix is smaller and cheaper to maintain.

The implementation's scalability limits are real and should be stated plainly. `_item_based_cf`` loads every `confirmed_paid`` order on every request and rebuilds the full co-occurrence map in memory, which is $O(\text{orders} \times \text{items-per-order}^2)$ per call. This is acceptable for a seeded demonstration and untenable at any real volume.

The correct production form is to precompute the item-item matrix offline on a schedule, persist it, and have the request path perform only lookup and aggregation — which is exactly what makes item-based attractive in the first place, and which the current implementation fails to exploit. Retrieval at larger scale would move to approximate nearest-neighbour search over embeddings, with the co-occurrence signal contributing to a ranking stage rather than being the retrieval mechanism.

Q27. Every recommendation carries an explanation tag. Is this a usability feature, an ethical requirement, or both?

Answer. Both, and the two motivations reinforce each other rather than competing.

As usability, it addresses Norman's gulf of evaluation: a user seeing an unexplained list cannot tell why these items appeared and therefore cannot calibrate trust in the list or correct it. 'Because you viewed running shoes' tells the user both why the item is there and, implicitly, what the system believes about them — which lets them discount a recommendation founded on an accidental click.

As ethics, it implements the transparency principle from Unit 11.6: the privacy-personalisation paradox is that users value personalisation while distrusting the surveillance producing it, and transparency about why an item is shown is the primary trust-preserving response, alongside user controls and data minimisation.

In MAUMart it is specified as a functional requirement rather than a design preference, which is the substantive point. `ListingOut` carries a `reason` field, every recommendation path sets it before returning, and the client renders it. That makes it testable — the test suite asserts that every returned recommendation has a non-empty reason — and therefore enforceable rather than aspirational.

The related requirement is that sponsored placements must be labelled 'Sponsored' and never blended into an organic explanation tag. That is the point at which transparency stops being a nicety: presenting paid placement as algorithmic relevance is a misrepresentation, and the labelling rule is what prevents the recommendation surface from quietly becoming an unmarked advertising channel.

Q28. What is popularity bias, how does this implementation risk it, and what would mitigate it?

Answer. Popularity bias is a feedback loop in which already-popular items receive more exposure, accumulate more interactions as a result, and are therefore ranked higher still. The consequence is that the effective catalogue narrows: new and niche items never accumulate the interactions that would let them be recommended, regardless of their quality or fit.

This implementation risks it in two places. The cold-start tier ranks purely by `sales\_count`, so the items already selling best occupy every slot a new user ever sees. The collaborative tier derives scores from co-purchase counts, which are themselves a product of past exposure, so the bias compounds rather than being corrected at the next tier.

Mitigations, roughly in order of effort. Exploration: reserve a proportion of result slots for items with little interaction history, or formalise the trade-off with contextual bandits so the system balances exploiting known preferences against acquiring information — which is what keeps it learning rather than merely repeating itself. Popularity discounting: divide or dampen scores by a function of overall item popularity so that co-occurrence with a ubiquitous item counts for less.

Diversity-aware re-ranking: after scoring, penalise results too similar to those already selected, which raises category and vendor spread. Content-based blending for new items, so an item with good metadata can be recommended before it has any interactions at all.

For a campus marketplace the commercial stake is direct: a new student vendor who receives no impressions will not persist on the platform, so unmitigated popularity bias suppresses the supply side the marketplace depends on.

Section F: Security Implementation

Q29. Explain how passwords are handled, and why bcrypt is used rather than SHA-256.

Answer. Registration passes the plaintext password to `hash_password`, which calls `bcrypt.hashpw` with a freshly generated salt and stores only the resulting hash. Login calls `bcrypt.checkpw` against the stored hash. The plaintext is never stored, never logged, and exists only for the duration of the request.

SHA-256 is unsuitable despite being cryptographically sound as a hash, and the reason is that it is designed to be fast. Speed is a virtue for integrity checking and a defect for password storage: an attacker with a leaked hash table can compute billions of SHA-256 candidates per second on commodity GPU hardware, so a stolen database of SHA-256 password hashes is effectively a database of passwords for any user whose password is not extremely strong.

bcrypt is deliberately slow and, critically, has a tunable cost factor, so the work per guess can be raised as hardware improves. It also incorporates a per-password random salt automatically, which defeats precomputed rainbow tables and ensures two users with the same password have different hashes.

The general principle is that password hashing requires a purpose-built slow function — bcrypt, scrypt or Argon2 — rather than a general-purpose fast one. Argon2 is the current recommendation for new systems as it additionally resists GPU and ASIC parallelism through memory-hardness; bcrypt remains acceptable and is what this project uses.

The same reasoning explains why reset tokens are hashed with SHA-256 instead: those are high-entropy random values, so brute force is infeasible regardless of hash speed, and slow hashing would buy nothing.

Q30. Describe the session mechanism, its expiry behaviour, and its principal weakness.

Answer. On successful login or registration, `create_session` generates a 24-byte random hex token via `secrets.token_hex` and stores it in a module-level dictionary mapping the token to the user identifier and an expiry timestamp 24 hours ahead. The token is returned to the client, which sends it as `Authorization: Bearer <token>` on subsequent requests.

`get_current_user` looks the token up, deletes it and raises 401 if expired, and otherwise loads and returns the user. `create_session` also prunes expired entries opportunistically, so the dictionary does not grow without bound in a long-running process. `invalidate_session` supports logout, and `invalidate_all_sessions_for_user` is invoked on password reset and on account deletion, so both operations actually evict existing sessions rather than merely changing a stored value.

`secrets` rather than `random` is a deliberate choice: the latter is a deterministic pseudo-random generator whose output is predictable from observed values, which would be catastrophic for session tokens.

The principal weakness is that the store is in process memory. The application is therefore not horizontally scalable as written — two instances behind a load balancer would each hold different sessions, so a user's requests would authenticate or fail depending on which instance received them. A restart logs every user out. This directly contradicts the stateless application tier the architecture claims, and is the single most significant gap between the stated design and the implementation.

The correction is either a shared session store such as Redis, which the architecture already anticipates, or stateless signed tokens such as JWTs — noting that JWTs trade the revocation problem back in, since a signed token remains valid until expiry unless a denylist is maintained.

Q31. Explain the rate limiter's algorithm, the parameters chosen, and its limitations.

Answer. `rate_limit(max_attempts, window_seconds)` is a dependency factory returning a checker. The checker composes a key from the request path and client address, prunes timestamps older than the window from a per-key list, raises 429 if the remaining count has reached the maximum, and otherwise

appends the current timestamp. This is a sliding-window log: unlike a fixed window, it cannot be defeated by concentrating a burst at a window boundary.

It is applied to `/auth/login` and `/auth/register` at twenty attempts per five minutes, and to `/auth/password-reset/request` at ten per fifteen minutes. The purpose is to make online brute-force and credential-stuffing attacks impractical while remaining well above legitimate use — a genuine user mistyping a password several times is unaffected. The reset endpoint is limited more tightly because it triggers outbound email and is therefore also an abuse vector for spamming a third party.

Limitations, which matter more than the mechanism. It is per-process, so multiple instances each enforce their own limit and the effective ceiling multiplies by instance count. It is keyed on client address, so an attacker distributing across many addresses evades it while many legitimate users behind one NAT or proxy share a bucket unfairly. It holds state in memory, so a restart clears all counters. And it is not applied to the rest of the API at all, leaving scraping and enumeration of listings unthrottled.

The production form is a shared counter in Redis keyed on a combination of address and account identifier, with progressive delays or account lockout as a second layer, and a WAF or edge rate limiter absorbing volumetric attacks before they reach the application.

Q32. Explain webhook signature verification: the threat it addresses and the implementation.

Answer. The threat is forged notifications. A payment webhook endpoint must be publicly reachable for the gateway to call it, which means anyone on the Internet can also call it. Without verification, an attacker can POST a JSON body claiming `charge.success` for an arbitrary reference and cause the server to mark an unpaid order as paid and release goods. This is the single highest-value attack against a naive e-commerce integration.

Paystack signs the raw request body with HMAC-SHA512 keyed on the merchant's secret key and sends the result in an `x-paystack-signature` header. `verify_webhook_signature` recomputes that HMAC over the received bytes and compares it to the header using `hmac.compare_digest`. Only a party holding the secret can produce a valid signature for a given body, so a forged payload fails.

Three implementation details carry the security. The comparison must be constant-time — `compare_digest` rather than `==` — because an early-exit comparison leaks information through timing that can be exploited to forge a signature byte by byte. The HMAC must be computed over the exact raw bytes received, not over a re-serialised parsed body, since re-serialisation changes whitespace and key order and invalidates the signature. And an absent header must be rejected rather than skipped, which the implementation does explicitly.

This was verified with a dedicated test asserting five properties: a correct signature is accepted, and a wrong signature, an empty signature, a tampered body and a wrong secret are each rejected. That verification required no network access, which is why it could be done honestly in an environment with no route to the provider. An important qualification: the function is tested but is not yet called by any route. The application has no webhook endpoint, so in live mode this protection would not be in effect until one is built.

Q33. Enumerate the security controls implemented and identify what remains unaddressed.

Answer. Implemented. Authentication: bcrypt password hashing with per-password salts; cryptographically random session tokens with 24-hour expiry; session invalidation on logout, password reset and account deletion. Authorisation: role-based dependencies (`require_role`) plus per-resource ownership checks, so a vendor can update only their own orders and listings.

Input handling: Pydantic schema validation at every boundary rejecting malformed bodies before handler logic; SQLAlchemy parameterised queries structurally preventing SQL injection. Abuse resistance: sliding-window rate limiting on authentication endpoints; generic responses on password reset to prevent account enumeration.

Transport and browser: an explicit CORS origin allow-list replacing a wildcard; `X-Content-Type-Options`, `X-Frame-Options` and `Referrer-Policy` headers; verified TLS operation under uvicorn with a certificate. Payment: idempotency-keyed charging; HMAC-SHA512 webhook signature verification; no card data stored, PCI scope delegated to the gateway. Supply chain: dependency vulnerability scanning with `pip-audit`.

Unaddressed, stated plainly. No penetration test has been conducted, so the above is a list of controls rather than evidence of resistance. Sessions and rate limits are per-process, so both weaken under horizontal scaling. No account lockout or progressive delay supplements rate limiting. No Content-Security-Policy header, which requires the deployed frontend's origins to set correctly. No audit log of privileged admin actions — approvals, refunds and dispute resolutions are not independently recorded, which is a genuine gap for a system handling money. No two-factor authentication. No file upload validation, though this is currently moot because listings accept photo URLs rather than uploads — adding real uploads would introduce a new attack surface requiring content-type and content validation.

Email verification exists and is tested but gates nothing, which is an unmade product decision rather than a defect.

Section G: Testing and Quality Assurance

Q34. Describe the testing strategy, identifying what each suite verifies and what it cannot.

Answer. Five suites, each with a bounded claim. `tests/test_api.py` contains twenty-nine end-to-end cases against a running server covering registration and login validation, the vendor verification gate, restricted-category routing, checkout with stock and overselling checks, the order status machine including a rejected backwards transition, verified-purchase review constraints, admin approvals, the NDPA data-rights endpoints, VAT computation, password reset, email verification and refund execution. It verifies system behaviour through the API; it cannot verify anything about the user interface.

`tests/test_payment_gateway.py` verifies webhook signature handling across five cases. Pure computation, no network — which is why it is trustworthy in an environment with no provider access. `tests/test_notifications.py` verifies SMTP delivery against a local `aiosmtpd` server, checking recipient, subject and body over a genuine protocol exchange. It verifies the client speaks SMTP correctly; it cannot verify delivery to a real inbox.

`tests/test_rate_limit.py` verifies that twenty failed logins return 401 and subsequent attempts return 429. It must run against a freshly started server, since it deliberately exhausts the bucket. `tests/test_accessibility.py` parses the rendered storefront HTML for missing alt text, heading-level gaps, the `lang` attribute and unlabelled form inputs across four page types.

`frontend/smoke_test.py` constructs all eleven Flet screens against the live backend with a fake page object. This is the suite whose limits most need stating: it verifies that every view builds without raising, and verifies nothing whatsoever about layout, visual correctness, interaction or usability.

The honest summary is that the backend is well covered behaviourally but was not covered under concurrency or load, the client is covered only structurally, and no penetration or assistive-technology testing has been performed. A limited concurrency and load measurement was made afterwards: it found overselling under simultaneous checkout and multi-second recommendation latency at 5,000 orders, neither of which the functional suite could detect.

Q35. The frontend smoke test uses a fake page object rather than a real Flet runtime. Justify this and state its limits.

Answer. The justification is environmental and practical. Driving a real Flet application requires a Flutter runtime and a display, neither available in the development environment. Rather than skip client testing entirely, the smoke test substitutes a minimal `FakePage` implementing the surface the view builders actually touch — `clean`, `add` and `update` — and calls each view builder with it, a real `AppState` and a no-op `navigate` function.

What this catches is substantial and was not hypothetical. The suite found nine distinct API mismatches in one run when the project was written against an older Flet idiom than version 0.86 provides: `content=` replacing `text=` on button constructors, `ft.Border.all()` replacing `ft.border.all()`, `ft.BoxFit` replacing `ft.ImageFit`, and `ft.Alignment.CENTER` replacing `ft.alignment.center`. It also exercises the real API underneath, so a broken endpoint or a changed response shape surfaces as a failing view build.

What it cannot catch, and this must be stated rather than implied. It does not verify that anything renders correctly, that layout is sensible, that text is legible, that controls are reachable, that interactions produce the intended effect, or that the interface is usable at all. A view that constructs a control tree with every element positioned off-screen passes.

The appropriate characterisation is that this is a structural or smoke test — it establishes that the code path is not broken — and that it is a complement to, not a substitute for, manual walkthrough and usability testing. Presenting it as evidence of interface quality would be the failure mode described in the testing-evidence discussion: claiming more than the test supports.

Q36. Two defects were found during testing rather than by inspection. Describe them and state what they illustrate.

Answer. The first was a missing SQLAlchemy relationship. ``Order`` carried a ``vendor_id`` foreign key but no corresponding ``vendor`` relationship attribute, while ``orders.py`` accessed ``order.vendor.business_name`` when serialising responses. Every vendor-facing order endpoint therefore returned 500. It was invisible to inspection because the foreign key was present and the code reads naturally; only executing the path revealed that the attribute did not exist.

The second was an XML escaping failure in the generated sitemap. Category names containing an ampersand — 'Books & Study Materials' — were interpolated into ``<loc>`` elements without escaping, producing invalid XML. This was caught not by looking at the output, which appears plausible, but by parsing it with ``xml.etree.ElementTree``, which raised immediately. A search console would have rejected the sitemap silently, so the failure would have manifested as unexplained absence from search results rather than as an error.

What they jointly illustrate is that execution finds a class of defect that review does not. Both involved code that reads correctly and would survive a careful walkthrough; both failed on contact with real data or a real call.

The sitemap case carries the sharper lesson, and it generalises beyond this project: verifying an artefact by looking at it is weaker than verifying it with the parser that will actually consume it. The same principle underlies checking contrast ratios by computation rather than by eye, and parsing rendered HTML for accessibility properties rather than assuming the templates are correct.

Q37. What is the difference between unit, integration and end-to-end testing, and where does this project sit?

Answer. Unit testing exercises a single function or class in isolation, with dependencies replaced by test doubles. It is fast, precisely localises failures, and verifies logic rather than integration. Integration testing exercises components together — a handler with a real database, for instance — verifying that the seams between them work. End-to-end testing exercises the full stack through its external interface, verifying the system as a user or client encounters it.

This project sits predominantly at the end-to-end level. ``test_api.py`` starts a real server against a real database and makes real HTTP requests, asserting on status codes and response bodies. ``test_notifications.py`` is integration testing against a real local SMTP server. ``test_payment_gateway.py`` is genuine unit testing — it tests one pure function with no dependencies. The frontend smoke test is integration testing of view construction against a live backend.

The trade-off implied by that distribution is worth stating. End-to-end tests have high confidence value because they exercise what actually ships, but they are slower, require orchestration of server and database, and localise failures poorly — a failing case tells you the behaviour is wrong without telling you which layer is responsible.

The conventional guidance is the test pyramid: many fast unit tests, fewer integration tests, fewest end-to-end tests. This project inverts that, which is defensible for a coursework system where demonstrating correct behaviour matters more than fast feedback on a large codebase, but would not scale: as the system grew, the absence of unit coverage on the checkout costing logic, the recommendation scoring functions and the status transition table would make refactoring risky.

Section H: Deployment, Scalability and Limitations

Q38. Why does the project include a server-rendered storefront alongside the Flet application, and what does it contain?

Answer. Because Flutter's web target renders to a canvas rather than to semantic HTML in the document. A crawler fetching a Flet page therefore finds no product titles, prices or descriptions to index. For a marketplace, where organic search is a primary acquisition channel, an uncrawable catalogue is a structural defect rather than a cosmetic one.

The storefront is a FastAPI router using Jinja2 templates, served from the same process as the JSON API. It exposes home, category, product detail and search pages as genuine server-rendered HTML.

Product pages carry schema.org `Product` and `Offer` structured data as JSON-LD — name, description, category, price with `priceCurrency` of NGN, availability derived from stock, seller organisation, images, and `aggregateRating` where genuine reviews exist. This is the practical application of the knowledge-representation material: structured data is what makes a page eligible for rich results and shopping surfaces. Open Graph tags serve social share cards, `robots.txt` declares crawl policy and points to the sitemap, and `sitemap.xml` is generated dynamically over every category and live listing.

The split is by access pattern: public read-only catalogue pages are server-rendered and crawlable; authenticated actions — login, cart, checkout, and the buyer, vendor and admin dashboards — remain in the Flet client, linked from every storefront page. This mirrors how many production storefronts are actually built.

A secondary benefit worth noting: because these pages are real HTML, they can be statically analysed for accessibility properties — alt text, heading order, form labels, contrast — which a canvas-rendered application does not expose at all.

Q39. Assess the system's readiness for horizontal scaling, identifying every obstacle in the current implementation.

Answer. The architecture is designed for horizontal scaling and the implementation partially contradicts it, which is the honest assessment.

What supports it: the application tier holds no per-request state between calls; the database is shielded behind that tier; sessions are conceptually externalisable; the ORM makes the data tier replaceable with a server capable of replication; and the storefront's public pages are cacheable at a CDN because they are anonymous and read-only.

What blocks it, concretely. Sessions live in a module-level dictionary, so two instances hold disjoint session sets and a load-balanced request authenticates or fails depending on routing. The rate limiter holds its counters in the same way, so the effective limit multiplies by instance count. The mock payment gateway's idempotency record is likewise in-process, so a retry routed to a second instance would not be recognised as a replay.

Beyond state, the collaborative filtering routine rebuilds the entire co-occurrence map from all orders on every request, which is a per-instance CPU and memory cost that grows with order volume and would dominate response time long before scaling helped.

The remediation sequence is: move sessions, rate-limit counters and idempotency records to Redis, which the architecture already anticipates; precompute the recommendation matrix offline and reduce the request path to lookup; add database read replicas for catalogue traffic; then add instances behind a load balancer. Adding instances before the first step would produce a system that fails more confusingly than the single instance it replaced.

Only a limited measurement has been made: on a single-core sandbox with SQLite, browse latency reached a median of 2.4 seconds at 500 listings (507 SQL queries per request), the recommendation endpoint took about seven seconds per request at 5,000 orders and none of twenty requests completed

under ten concurrent users, and three simultaneous checkouts for one unit oversold it. Nothing has been measured on PostgreSQL, on multiple cores, or with multiple instances, so capacity claims beyond these findings still rest on architectural reasoning.

Q40. The project includes Dockerfiles, a Compose file and a CI workflow that have never been executed. Does that undermine them?

Answer. It limits what may be claimed about them without rendering them worthless, and the distinction is one an examiner is entitled to press on.

What can be claimed: they are written against the exact commands verified to work directly — the same ``uvicorn``, ``flet run --web`` and ``seed.py`` invocations used throughout development — and against documented Docker and GitHub Actions syntax. The backend image runs as a non-root user, includes a health check, and reads all configuration from environment variables. The Compose file orders startup with a Postgres health-check condition. The CI workflow runs the full suite against both SQLite and a real Postgres service container, plus the signature, notification and accessibility suites.

What cannot be claimed: that any of it works. No Docker daemon and no route to GitHub Actions existed in the development environment, so the first real ``docker compose up`` and the first real push are genuinely the first tests of these files. Configuration written correctly against documentation frequently still fails on first execution — a missing system library for ``psycopg2``, a path assumption, a service that starts before its dependency is truly ready.

The defensible position is to label them as unverified rather than to present them alongside tested components without distinction, which is what the project does.

One thing the exercise did produce that was real: writing the frontend Dockerfile exposed that the client hardcoded ``127.0.0.1:8000`` as its API base, which would silently fail inside a Docker network where the backend is reachable by service name. That was fixed to read ``MAUMART_API_BASE``. Going through the motions of a deployment path surfaced a genuine defect even without executing it.

Q41. State the most significant limitations of the implementation and the order in which you would address them.

Answer. Grouped by kind, since the remedies differ.

Correctness: the stock check and decrement are not atomic, permitting overselling under concurrent checkout; this was reproduced by measurement (three simultaneous checkouts for one remaining unit all succeeded and charged all three buyers), not merely inferred. One review per order is enforced by a query rather than a database constraint, so a concurrent double submission could pass both checks.

Verification: the live payment API has never been called, and the live-mode code path would in fact reject every payment as written (a successful Paystack initialisation returns a pending status that checkout treats as failure, and no webhook route exists); Docker and CI have never executed; there has been no penetration test, no production-scale load test (a limited measurement found serious defects, described elsewhere in this bank), and no assistive-technology accessibility testing on the Flet dashboards. The requirements themselves rest on reasoned inference rather than collected primary data, since the elicitation instruments exist but have not been run.

Scalability: in-process sessions, rate-limit counters and idempotency records prevent multi-instance deployment; the collaborative filtering routine recomputes from all orders per request.

Functional gaps: email verification gates nothing; there is no in-app buyer-vendor messaging, which the dispute process would benefit from; there is no audit log of privileged admin actions; listings accept photo URLs rather than uploads.

Priority order and the reasoning behind it. First, the stock race condition, because it is a correctness defect that silently takes money for goods that do not exist — the only fault here that actively harms a user. Second, externalise session and rate-limit state, because it is cheap and unblocks everything else.

Third, run the elicitation, because requirements resting on inference is the deepest weakness and no amount of engineering compensates for building the wrong thing. Fourth, complete a live payment integration test. Fifth, load testing, to convert scalability from argument into evidence. Deployment automation comes last, because reliably deploying an unverified system is not progress.

Q42. If asked to rebuild this system, what would you change architecturally?

Answer. Three changes, in order of impact.

First, split the client by access pattern from the outset rather than retrofitting. Flet is a good fit for the vendor and admin dashboards — authenticated, form-heavy, and genuinely benefiting from one Python codebase across desktop, web and mobile, with search visibility irrelevant. It is a poor fit for the public catalogue, for the crawlability reason. Designing that split at the start would have produced a cleaner system than adding a server-rendered layer after discovering the problem.

Second, build the non-functional requirements in from the beginning. Rate limiting, session expiry, structured logging, CORS restriction, the data-rights endpoints and the security headers were all added in hardening passes after the functional system worked, and several required revisiting code that would have been simpler to write correctly once. This is precisely what the literature predicts — non-functional requirements are cheap at design time and expensive as retrofits — and doing it in the wrong order demonstrated it at first hand.

Third, externalise state from the start. The in-process session store is the single design decision most at odds with the architecture the report describes, and it exists only because it was the fastest thing to write. Using Redis from the beginning would have cost little and would have made the stateless-tier claim true rather than aspirational.

A fourth, smaller change: unit-test the pure logic — checkout costing, recommendation scoring, the transition table — rather than relying almost entirely on end-to-end tests. The current distribution gives good confidence and poor failure localisation, which would become a real obstacle as the codebase grew.

Q43. How would you extend this system into a genuine production deployment? Outline the work and its sequence.

Answer. Five phases, sequenced so each depends only on what precedes it.

Phase one, correctness and verification. Fix the stock race with an atomic conditional update and reserve-before-charge sequencing. Add the missing database constraints. Complete a live-mode payment integration against the provider's test environment, including a real webhook endpoint calling the signature verification already implemented, with a verify-endpoint fallback. Run the elicitation instruments so the requirements have an empirical basis, and revise the SRS where findings contradict expectations.

Phase two, operational readiness. Move sessions, rate-limit counters and idempotency records to Redis. Deploy PostgreSQL with replication and confirmed backup and restore procedure — the scripts exist and the cycle has been tested, so this is scheduling and monitoring rather than new work. Terminate TLS at a reverse proxy with a CA-issued certificate and add HSTS there. Wire structured logs to an aggregator and add metrics, alerting and an on-call path.

Phase three, compliance and governance. Obtain legal review of the four policy documents, whose placeholders are deliberately unresolved. Add an audit log of privileged admin actions. Conduct a data protection impact assessment and complete any NDPC registration obligations. Decide and implement what email verification actually gates.

Phase four, scale and quality. Precompute the recommendation matrix offline. Introduce a dedicated search service if catalogue growth warrants it. Load test to establish real capacity. Commission an external penetration test and a genuine accessibility audit with assistive technology.

Phase five, product. In-app buyer-vendor messaging, real image uploads with content validation, vendor payouts and reconciliation, and vernacular language support — deferred deliberately, because none of it matters if the system in phases one and two is not sound.